Program:

~~class commandline~~

{—

~~public static vo~~

System.out.println("sum is :" +res);

}

}

- Garbage is acces by finalized keyword.

→ 3rd chapter Interface, packages, Exceptions

Interfaces:

- An interface is a pure Abstract class which contains only abstract methods and final variables.
- To the interface object cannot be intialized because.
- To make the interface available outside the package any where then the access specifier of the interface should be public.
- The interface and its methods are by default Abstract therefore, they should not declare with the keyboard abstract.
- All variables defined in the interface are by default public static and final variables.

∴ they can be accessed using the interface name.

- Interface are used for implementing into subclasses and it is the responsibility of subclass

that it should ~~overriding~~ override all abstract method of interfaces. otherwise the subclass also becomes an abstract class.

- Object can be Instanciated to these subclasses to make use of them.

Multiple Inheritance through interface

In Java,

- Multiple inheritance cannot ~~using~~ Implemented using class whereas it can be achieved using interface.
- A class can implement any no. of interface which is said to be multiple inheritance.

Extending Superclass and implementing Interface

It is possible that a subclass can extend superclass and Implement interface that superclass Extend first and then interface.

Program:-

```
//extending class and implementing interface
interface inter 1
{
void sum(int x, int y);
}
class math1
{
public void sub(int x, int y)
{
System.out.println("\n sub = " + (x-y));
}
}
class myclass extends math1 implements inter
{
public void sum(int x, int y)
{
System.out.println("\n sum = " + (x+y));
}
}
class in3
public static void main (String[] args)
{
myclass obj = new myclass();
obj.sum(10,20);
obj.sub(20,10);
}
}
```

5M

Extending Interface

similar to class has interface can be extend another interface.

- The inheritaned interface contains all abstract method are it's super interface.
- Any class that implements inheritaned interface should implement all abstract method of sub and super interfaces.

Program:-

```
interface inter1
{void sum(int x, int y);
```

```
}
interface inter2 extends inter1
{
  void sub(int x, int y);
}
class myclass implements inter2
{
  public void sum(int x, int y)
  {
    System.out.println("\n Sum = "+(x+y));
  }
  public void sub(int x, int y);
  {
    System.out.println("\n sub = "+(x+y));
  }
  public void sub(int x, int y)
  {
    System.out.println("\n sub = "+(x-y));
  }
}
class in4
{
  public static void main(String[] args)
  {
    myclass obj = new myclass();
    obj.sum(10, 20);
    obj.sub(5, 2);
  }
}
```

* Variables in Interface:

- Variables created in interfaces are by default public, static and final. therefore these variables should be intiallized and cannot be modified.
- These variables can be accessed without using object, but can be accessed using interface name are implementing class name.

program:

```
interface inter1
{
  int A = 100;
}
class myclass implements inter1
{
  public void show()
  {
    S.O.P("\n value of A = "+A);
  }
}
class in5
{
  PSVM(String[] args)
  {
    myclass obj = new myclass();
    obj.show();
    S.O.P("\n inter1.A = "+inter1.A);
    S.O.P("\n myclass.A = "+myclass.A);
  }
}
```

* Nested Interfaces:

- An interfaces can be declared as a number of the class (or) other Interface is known as nested interface.
- Nested interface should be accessed the outside that class or interface using the outer class name or interface name.

Program:

```
class A
{
  public interface Inter1
  {
    public void sum(int x, int y);
  }
class B implements A.Inter1
  {
    pv sum (int x, int y)
    {
      S.O.P ("\n Sum = " + (x+y));
    }
}
class in 7
{
  ps vm (String [] args)
  {
    B obj = new B();
      obj.sum (5,8);
  }
}
```

** Packages:- → define a package

- package is a container of class.
- package eliminate name space collision.
- The class in packages should be public
- The class in package should be public otherwise the class cannot be accessed outside the package
- the package statement should be first statement in the Java program.
- packages are created using the package keyword.

program:

```
package pack1;
public class math1
{
  public void sum(int x, int y)
  {
    System.out.println("\n sum = " + (x+y));
  }
  public void square (int x)
  {
    System.out.println("\n square = " + (x*x));
  }
}
```

Importing package classes: The classes present in package can be used in other program by importing the classes of packages in other Java programs

Syntax: `import package name . class name ;`

(or)

`import package name . * ;`

program:

```
import pack1.math1;
class m1
{
  public static void main (String args[])
  {
    pack1.math1 obj = new pack1.math1();
      obj.sum(5,60);
      obj.square(5);
  }
}
```

Access protection:-

There are 4 type of access specifier in java

- Those are ~~compile~~ public, private, protected and default

1) private can't be accessed. (or) private member can't be accessed in ~~public~~ sub class of same package whereas default, protected public can be accessed

2) The private members can't be accessed in non sub-class of same package whereas default protected, public can be accessed

3) private, default and protected can't accessed in non-subclass of different packages whereas Public can be accessed.

4) private & default members can't accessed in sub-class of different packages whereas protected & public can accessed.

- The visiblity of this access specific in different classes in packages are shown in the below table

	Private	Default	Protected	Public
same class	yes	yes	yes	yes
Same package non-subclass	No	yes	yes	yes
same package sub-class	No	yes	yes	yes
Different package non-sub class	No	No	No	yes
Different package sub class	No	No	yes	yes

Programs:-

~~(Same class the)~~

① Same package and non-subclass:-

```
Package Samplepack;
public class Sample
{
    private int x=10;
            y=20; // deafault access
    protected int z=30;
    public int p=40;
}
```

```
Package Samplepack;
public class myclass
{
    public void show()
    {
        Sample obj = new Sample();
        System.out.println("\n x.value = " + obj.x);
        System.out.println("\n y value = " + obj.y);
        System.out.println("\n z value = " + obj.z);
        System.out.println("\n p value = " + obj.p);
    }
}
```

```
import Samplepack.myclass;
class A
{
    public static void main(String args[])
    {
        Samplepack.myclass obj = new Samplepack.myclass();
        obj.show();
    }
}
```

② Same package sub-class:-

```
package Samplepack;
public class Sample
{
    private int x=10;
            y=20;
    protected int p=40;
}
```

```
package Samplepack;
public class myclass extends Sample
{
    public void show()
```

```
}
System.out.println("In x values = " + x);
System.out.println("In y value = " + y);
System.out.println("In z value = " + z);
System.out.println("In P value = " + P);
}
}
import Samplepack.myclass;
class A
{
public static void main(String[] args)
{
Samplepack.myclass obj = new Samplepack.myclass();
obj.show();
}
}
```

③ Different package and non-sub class

- if the class are present in the different pacakage and one class is not a sub-class of other now in this situation inside public class sample the non-sub class default & protected member can't be accessed and otherwise public can be accessed
- The private members however can't be accessed outside the class
- The following program demonstrate the same

Program:-

```
package Samplepack;
public class Sample
{
private int x = 10;
y = 20;
protected int z = 30;
public int P = 40;
}
package Samplepack1;
import Samplepack.Sample;
public class myclass1 extends
{
public void show()
{
Sample pack.sample obj = new Samplepack.Sample();
System.out.println("In value of x is = " + obj.x); //error
System.out.println("In value of y is = " + obj.y); //error
System.out.println("In value of z is = " + obj.z); //error
System.out.println("In value of P is = " + obj.P);
}
import Samplepack1.myclass1;
public class B
{
public static void main(String args[])
{
Samplepack1.myclass1 obj = new Samplepack1.myclass1();
obj.show();
}
```

(4) Different package and Sub-class

- If Super and sub class are present in different package in this situation inside the sub-class defaullt can't be accessed whereas, protected and public can be accessed.
- private of Superclass can't be accessed outside the class.

program:-

```
package Samplepack;
public class Sample
{
  private int x=10;
          y=20
  protected int z=30;
  public int p=40;
}
package samplepack;
import Samplepack.*;
public class myclass1 extends Samplepack.Sample
{
   Public void Show()
   {
System.out.println("In value of x is = "+x);
System.out.println("In value of y is = "+y);
System.out.println("In value of z is = "+z);
System.out.println("In value of p is = "+p);
}
import Samplepack1.myclass1;
public class B
{ public static void main(String args[])
{
Samplepack1.myclass1 obj = new Samplepack1.myclass1()
   obj.show();
}
}
```

Exceptions :- (exception handling)

An exceptions in java is one of the buzzer word.

- Exceptions is an error or unexcepted event that occur during the execution of java program and interputs the normal flow of the program
- Handling of exceptions is known as exception Handling
- Exception Handling can be achieved in java using the keyword

⇒ try, catch, throw, throws, finally are keyword

Syntax:-

```
try
{
   : // error generated code
}
catch(exception type 1 ex1)
{
   : // Code to handle excetion type 1
}
catch(exception type 2 ex2)
{
   : // Code to handle exception type 2
}
finally
{
   : // final code to execute
}
```

try block :-

The try block is used to scan the code or exception during execution of program

- If exception occur the program flow interputs and control goes to catch block
- try block is used test the code for exception
- A try any number catch block and catch block should immediately follows the try block.
- Each catch block can handling one type of exception that is raised in try block
- The Exception raised in try block is caught by catch block and handle it so that the program doesn't terminate abnormally

finally block :-

The finally block execution if exception raised or not raised in try try block

- The finally block should be the last block of try
- A try can contain any no. of catch block but should contain only 1 finally block.

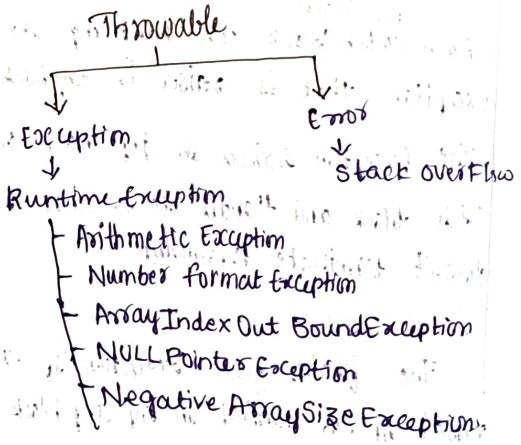
Therefore the try block should contain either catch or finally or both

* Exception are predefined class in java which are present in "java.lang"

* The superclass most class for an exception class is "Throwable"

This class is inherited into exception & error class

- The Exception class further inherited into run-time exception
- The run-time exception having sub-class as follow



Program:- //Abnormal program termination

```
class ex1
{
public void main(String args[])
 {
   int a, b, c;
   a = Integer.parseInt(args[0]);
   b = Integer.parseInt(args[1]);
   c = a/b;
 System.out.println("value of c="+c);
 System.out.println("program terminates successfully...");
 }
}
```

compiler: javac filename
java ex1 10 2
Java ex1 10 0 val of c=I
program terminated successfully

1. program to demonstrate Arithmetic Exception

```
class ex2
 {
 public static void main(String args[])
 {
   int a, b, c;
   a = Integer.parseInt(arg[0]);
   b = Integer.parseInt(arg[1]);
```

- javac ex2.java
- java ex2 10 5 → value give

```
   try
    {
     c = a/b;
     System.out.println("c = +", c);
    }
   catch(ArithmeticException e)
    {
 System.out.println("\n Error: Divided by zero");
 System.out.println("\n e ;" +e);
 }
```

System.out.println("program terminated");
}
}

• In case a=10, b=0 in which 10 is divisible by 0 which is an error.
∴ JVM created an object of Arithmetic exception class and object is stored with error message.
• Finally JVM throw an a Arithmetic exception object which is catight by catch block and the exception object direct address is assign to e and catch block is executed after execution of catch block. remaining statement till it reach to main and program terminates now.

2. Array Index Out of Bound Exception:-

• Program to demonstate ArrayIndex Out of Bound Exception

```
class ex3
{
public static void main(String args[])
{
 try
 {
 System.out.println("In first argument : "+arg[0]);
System.out.println("In second argument : "+arg[1]);
System.out.println("In Third argument : "+arg[2]);
}
Catch(ArrayIndexOutOfBoundException e)
{
System.out.println("In Index out of ranges");
System.out.println("In error message : "+e);
}
System.out.println("program terminates");
}
}
```

> javac ex3.java error message : java.lang.ArrayIndexOutOfBoundsException
> java ex3 10 20

• In the statement System.out.println third argument into the array arg does n't contain index[2] :
∴ JVM identify has error and creates the object of ArrayIndexOutOfBoundException throw, which is catight by catch block and print the error message as " A Java.lang.ArrayIndexOutOfBoundException : 2"

Multiple Catch block :-
A try can have any no. of catch blocks and each catch block can handle only one exception

Syntax:-

```
try
{
 // code to scan for exception
}
catch (Exceptiontype1 e1)
{
 // code to handle exceptiontype1
}
 catch (Exceptionty2 e2)
{
 // code to handle exceptiontype2
}
finally
{
 // final code to execute
```

Program:

```
class mcatch
{
public static void main(String args[])
{
  int a=0, b=0, c=0;
   try
   {
    a= Integer.parseInt(args[0]);
    b= Inger.ParseInt(args[1]);
     c= a/b ;
  System.out.println(" in c= "+c);
  }
Catch( NumberFormatException e1)
  {
System.out.print(" in error1: "+e1);
  }
Catch(ArithmeticException e2)
{
System.out.print(" in Error: "+e2);
  }
catch(IndexOutOfBoundsException e3)
{
 System.out.println(" in error3 :"+e3);
}
System.out.println(" in program terminates.");
}
}
```

Handling Multiple Exception:-

Syntax:-

- It is possible to handle all exception by example in a single catch block. catch block with Exception type can handle ~~because~~ All runtime exception because all runtime exceptions are subclasses of Exception

Syntax:-

```
try
{
// code to handle exception
.
.
}
Catch(Exception e)
{ // handle a runtime exception
}
```

Program

```
class mcatch1
{
public static void main(String args[])
{ int a=0, b=0 , c=0;
try
{
a= Integer.parseInt(args[0]);
b= Integer.parseInt(args[1]);
c= a/b ;
System.out.println(" in c= "+c);
}
catch(Exception e)
{ System.out.println("error:"+e);
```

Note:

```
try
{
  ==
}
catch(NumberFormatException e1)
{
  ==
}
catch(ArithmeticException e2)
{
  ==
}
catch(Exception e3)
{
  ==
}
```

- When catch block with exception type is used with another catch block then catch block with Exception type should be last catch block
- In above catch block with Exception type handle all exception other than numberformat exception & Arithmetic exception
- But the following is not correct

Syntax:-

```
try
{
  // code handle exception
}
catch (Exception e1)
{
  // handle all runtime exception
}
catch (ArithmeticException e2)
{
  ==
}
```

Nested try

- A try can be written in another try is known as nested try.
- Exception raise in outer block can be handle wider outer try catch. whereas exception raised in inner try can be handle by inner try catch block. otherwise, the exception are raised to outter try catch block can be handle by outer try block.

Program:

```
class nesttry
{
public static void main(String arg[])
{
  int a = 0, b=0, c=0;
  try
  {
  a = Integer.parseInt(arg[0]);
      try
      {
        b = Integer.parseInt(arg[1]);
        c = a/b;
        System.out.print("\n c = "+c);
      }
    catch(ArithmeticException e1)
    {
    System.out.println("\n Error1: "+e1);
    }
  }
catch(NumberFormatException e2)
{
  System.out.println("\n Error2: "+e2);
}
```

```
Catch( ArrayIndexOutOfBoundException e3
{
System.out.println("\n Error3 : "+e3);
}
System.out.println("\n program terminate...");
}
}
```

Throwing Exception Manually :-

- In the previous program, exception are thrown by a JVM internally basis on the type of exception raised
- Java also allow to throw the exception explicitly in the program using throw keyword.

Syntax :-

Throw ThrowableInstance

- In the syntax ThrowableInstance means it should be an object of throwable or its sub classes
- The throw keyword throw the exception object explicitly

program:

```
class throw1
{
public static void main(String args[])
{
int a=0, b=0, c=0;
try
{
a = Integer.parseInt(args[0]);
b = Integer.parseInt(args[1]);
if(b==0) throw new ArithmeticException("divide by zero");
c = a/b;
System.out.println("\n c= "+c);
}
Catch(ArithmeticException e)
{
System.out.println("\n Error = "+e);
}
System.out.println("\n program terminates...");
}
}
```

Finally block

- A try should contain either catch or finally block
- The try can contain any no. of catch block but only one finally block
- When an exception raises in try blocks the concerned catch block execute otherwise the catch block doesn't execute, whereas finally executes if an exception raises or not or any other abnormal situations [i.e the statements in finally block execute compulsary]

Type of Exception

Exception class are of two types 1) Checked Exception
2) Unchecked Exception

- All java methods by default have permission to throw unchecked exception. When the java compiler come

across unchecked exception in a method then, compiler doesn't check any condition, wheather the method can throw the exception or not

- therefore called unchecked exception
- unchecked exception need not be included in any methods throws

~~Synat~~ program:-

```
Class Main
{
public static void main(String[] args)
{
  int a = 10;
  int b = 0;
   int c = a/b;
   System.out.println("Result = "+c);
}
}
```

output:-

Exception in thread "main"

java.lang.ArithmeticException: / by zero.

- It need not required to be handled by try-catch or declare using throws

nothing

Checked exception:-

The java method don't have permission to throw the checked exception by default. When the compiler comes across checked exception in the method it check the condition whether the method can throw the exception or not. is called checked exception.

- The checked exception need be included in the method throws list otherwise compiler will be given error
- The checked exception in the following table.

Check Exception	Description
IOException	occur during input/output operations
FileNotFoundException	occurs when a file is not found
SQLException	occurs during database operations.
ClassNotFoundException	occur when a class is not found
InterruptedException	occurs when a thread is interrupted.
ParseException	occurs when parsing fails
MalformedURLException	occur when a URL is invalid
NoSuchMethodException	occur when a specified method is not found
InstantiationException	occurs when an object cannot be instantiated.
IllegalAccessException	occurs when access to a class or member is not allowed.

Program:

```
class checkexp
{
public static void main(String argv[]) throws ClassNotFoundException
{
System.out.print("\n Sample program on checked exception.");
try
{
throw new ClassNotFoundException("class not found");
}
catch (ClassNotException e)
{
System.out.println("\n error : "+e);
}
System.out.print("\n program terminated");
}
}
```

Userdefined

- to String method:-

toString() converts an object's information into readable string representation. printing meaningful object details

Program:-

```
class Student
{
    String name;
     int age;
  Student(String name, int age)
  {  this.name = name;
     this.age = age;
```

```
}
@Override
  Public String toString()
{
  return "Student{name = '" + name + "', age = " +
                    age + "}";
}
class A
{
public static void main(String args[])
{
  Student s = new Student ("Rahul", 20)
  System.out.println(s);
}
}
```

* Userdefine exception

- Java allows programmer to created a new exceptions which are called userdefine exception
- userclefined class should satisfy the following condition to behave like an exception class
* User defined exception class should extend from exception class
* User defined exception class should contain tostring method to return the error message

Note:

* User defined exception classes are checked exception they should be listed with throws keyword

- A program demonstrate creation of user define exception.
- Write a program to create user defined exception class to handle an exception while creation of bank account, the minimume disposite to create an account is ₹ 1000/-

```
class MinimumBalanceException extends Exception
{
MinimumBalanceException(String msg)
{
  super(msg);
}
}
class Main
{
public static void main(String[] args)
{
  int deposit = 500;
  try
    if (deposit < 1000)
    {
      throw new MinimumBalanceException("Deposit
            must be at least Rs.1000");
    }
System.out.println("Account created successfully");
}
catch(MinimumBalanceException e)
  {
    System.out.println(e.getMessage());
  }
}
```

Unit : IV

Multi Threading

- extends Thread class
- Implements Runnable interface

Java.lang.Thread:

Java.lang.Thread is a class in java used to create and execute threads.

. A thread is small unit of execution within a program.

. Thread belongs to the java.lang package, which java makes available automatically

Program:-

```
class Main extends Thread
{
  public void run()
  {
    System.out.println("Thread is running");
  }
  public static void main(String[] args)
  {
    Main t = new Main();
    t.start();
  }
}
```

Thread method :-

start() :- Starts a new thread

run() : Contains the thread's task

sleep() : Pauses the current thread temporarily

join() : waits for another thread to finish.

isAlive() : checks whether a thread is still running.

Main thread:

the main thread is created automatically by the JVM when a java program starts. It executes the main() method

Program:

```
class main
{
public static void main(String[] args)
{
Thread t = Thread.currentThread();
System.out.println("current thread:" + t.getName());
System.out.println("Main thread is running");
}
}
```

- It starts execution from the main method
- Thread.currentThread() returns the currently executing thread
- .getName() Return the name of the thread

~~Creating of new threads:-~~

- Every java program has always at least one thread, even if you do not create any threads this thread is called the "main thread"
- the main thread is also know as the parent thread while the rest of the threads that are generated from it are known as child threads of the program
- Too When the main thread finishes the execution, the program terminates immediately
- Whenever the java program starts the main thread is created automatically. This main thread is available in all program.
- Controlling the Main thread
- sleep() Method : pauses a thread for a specified time. ex: Thread.sleep(3000);
- join(): Waits for another thread to finish
  Ex: studentThread.join();
- interrupt(): sends an interruption request to a thread. Ex: temporaryThread.interrupt();

Creating a thread

- There are two ways to create a thread in java.

1. By extending the thread class

- creates (create a child cl a class that extends the thread class
- Write the task to be performed by th thread override inside run().

- call the start() method to start the new thread
- Override the run() method in the child class
- the new thread begins executing the run() method

program

```
class My extends Thread
{
public void run()
{
System.out.println("hello");
}
public static void main(String[] args)
{
My h = new My();
h.start();
System.out.println("hai");
}
}
```

2) By implementing Runnable interface

- Create a class that implements the runnable interface.
- Override the run() method
- Create an object of the class
- Pass the object to the thread constructor
- Call the start() method to execute the thread

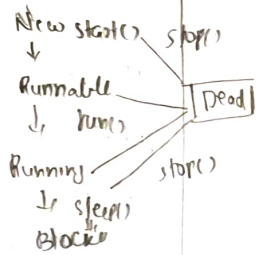
program:

```
class Hi implements Runnable
{
public void run()
{
System.out.println("hello Thread");
}
public static void main(String[] args)
{
Hi h = new Hi();
Thread t = new Thread(h);
t.start();
System.out.println("hai");
}
}
```

life cycle of a thread

- In java, a thread always exists in any one of following state. These state are

1. New
2. Runnable
- Running
- Blocked (non-runnable state)
- Dead



1 New state:

- The Thread object is created
- the thread is not yet started
- It is only the thread class

3 It moves to the runnable state

Runnable state

- the thread is ready to execute
- It enters this state after calling start()
- It waits for CPU time to execute.
- The thread may move to the running state when the scheduler select it.

Running state

- The thread executes its task
- It executes the code inside the run() method
- The CPU executes the threads instruction

4. Blocked / waiting state

- The thread waits for a resource or another thread.
- It cannot continue until the required condition is met.
- It may enter this state while waiting for input.
- After the condition is satisfied, it can become ~~Runn~~ runnable again

Terminate state

- The thread completes its execution
- It cannot be restarted using start()
- The thread reaches this state when run() finishes
- Once terminated, the thread cannot execute again.

Thread

- A thread is light-weight sub-process
- It is used to perform multiple task simultaneously
- Java supports multithreading
- Thread can created by Thread class and ~~implem~~ implementing the runnable interface.

Thread Priority:-

- The Priority of a thread describes how early it gets execution and selected by the thread scheduler
- In java, when we create a thread, a priority is assigned to it.
- The priority in which multiple thread are processed is called Thread priority.
- priority is given by the JVM or by the programmer explicitly
- The range of thread priority is from 1 to 10.
- getPriority(): return the thread priority
- setPriority(): sets the priority value.

- Default priority is 5
- minimum priority is 1
- Maximum priority is 10

Three constants in the Thread class

- MIN_PRIORITY = 1
- NORM_PRIORITY = 5
- MAX_PRIORITY = 10

program:

```
class Main
{
public static void main(String[] args)
{
Thread t = new Thread();
System.out.println("Default Priority" + t.getPriority());
t.setPriority(8);
System.out.println("New Priority:" + t.getPriority());
}
}
```

Multithreading.

Multithreading is process of executing multiple threads simultaneously within a single program.

- Java supports multithread through thread class java.thread allow us to create a lightweight process that executes some tasks
- We can create multiple threads in our program and start them. they use a shared memory area.
- It doesn't block the user because thread are independent and you can perform multiple operations at the same time.
- You can perform many operations together, so it save time.
- Threads are independent, so it doesn't affect other thread if an exception occur in a single thread.

isAlive()

- the isAlive() method of thread class tests if the thread is alive
- A thread is considered alive when the start() method of thread class has been called and the thread is not yet dead.
- This method return true if th thread is still running and not finished.

Syntax:

```
public final boolean isAlive()
```

• join()

• multithreading allows multiple threads to execute concurrently

• The join() method makes one thread wait for another thread to finish

• It helps maintain the required execution order

• The join() method may throw "InterruptedException"

• Joining thread in java has three functions

  • join()

  • join(long millis)

  • join(long millis, int nanos)

Syntax:-

• public final void join()

• public final void join(long millis, int nanos)

• public final void join(long millis)

• Synchronization

Synchronization in java is a process of controlling access to shared resources by multiple threads to avoid data inconsistency

• Synchronized prevents multiple threads from accessing a shared resource at the same time.

• Its avoid data inconsistency.

• It is achieved using the "synchronized" keyword

• It allows only one thread at a time to execute a synchronized method or block for the same lock.

• It helps maintain data consistency in multithreading

Type of Synchronization

1. Synchronized Method

  • The synchronized keyword is applied to a method

  • only one thread at a time can execute that method on the same object

Syntax:-

```
synchronized void display()
{
}
```

2. Synchronized block

  • The synchronized keyword is applied to a block of code

  • only the code inside the block is protected by the lock

Syntax:-

```
synchronized(this)
{
}
```

Program

```
class Test
synchronized void display()
{
System.out.println("Hello java");
}
public static void main(String[] args)
{ Test t = new Test
```

```
Thread t1 = new Thread(()-> t.display());
Thread t2 = new Thread (()-> t.display());
    t1.start();
    t2.start();
  }
}
```

Suspending and resume thread

- Suspending: Temporarily stopping the execution of a thread
- Resume:
  Allowing a suspended thread to continue execution
- suspend() is used to suspend a thread
- resume() is used to resume a suspended thread
- These method belong to the thread class
- Both methods are "deprecated" in java because they can cause deadlock.
- Modern java uses flags, locks, or other synchronization techniques instead

program

```
class Main
{
public static void main(String[] args)
throws InterruptedException
{
thread t = new Thread(()-> {
System.out.println("Thread is running");
});
  t.start();
t.join();
System.out.println("Thread completed");
```

Communication between thread (Inter-thread communication)

Inter-thread communication is a mechanism through which threads communicate with each other to coordinate their execution.

- It allows thread to communicatim with each other.
- It avoids unnecessary waiting between threads.
- Java provides three methods for communication
  - wait(): - makes a thread wait.
  - notify(): - wakes up one waiting thread.
  - notifyAll(): - wakes up all waiting threads.
- These methods belong to the object class.
- they must be called while holding the objects monitor (lock)

Program:

```
class Main
{
public static void main(String[] args)
{
Object obj = new Object();
synchronized(obj)
```

```
        §
            Obj.notify();
        System.out.println("Thread communication");
    }
}
```

wait() Method

- Makes the current thread wait until it is notified
- It belongs to the object class.
- It releases the object's lock while waiting
- It must be called inside a synchronized method or block.

Syntax :- `obj.wait();`

a. Notify()

- It is used for Inter-thread communication
- It belongs to the object class
- the must be called insid a

Syntax:

```
obj.notify();
```

- notifyAll()

Wakes up all thread waiting on the same objects monitor.

- The waiting threads compete to acquire the lock
- It belong to Object class
- It must be called inside

Syntax: `obj.notifyAll();`

Exploring java.io

java.io is a package in java used to perform input and output operations, such as reading data from a file and writing data to a file

- java.io provides classes for input and output operations
- It supports byte streams and character streams
- It provides classes to handles files and exceptions.
- It is used to read and write data from different source

classes

- File: Creates and manage files and directories
- FileInputStream: Reads bytes from a file
- FileOutputStream: Writes bytes to a file
- FileReader: Reads characters from a file
- FileWriter:- Writes characters to a file
- BufferedReader: Reads text efficiently

Ex:

```
import java.io.*;
    class Main
{
    public static void main(String[] args)
    {
    System.out.println("Exploring java.io");
    }
}
```

Exploring Java.util:

java.util is a package in a java that provides utility classes and interfaces for handling collections, dates, random numbers, and other operation

- It provides classes and interfaces for storing and managing data
- It supports collection such as lists, sets and map
- It provides classes for generating random numbers
- It support date and time utilities.

Classes

- ArrayList: stores elements in a resizable list
- Vector: stores elements in a dynamic array
- Stack: Follows LIFO (Last In, First out)
- HashMap: Stores data as key-value pair

Program:

```
import java.util.*;
class Main
{
public static void main (String[] args)
{
    ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        System.out.println(list);
}
```

Collection Framework

The collection framework is a set of classes and interface used to store and manipulated groups of object.

- It belongs mainly to the java.util package.
- It supports adding, removing, searching, and sorting element
- It provides interfaces such as list, set, and queue.
- It reduces programming effort and makes data handling easier

Interface	Example	Use
List	ArrayList, Vector	Allows duplicates
Set	TreeSet, HashSet	stores unique element
Queue	PriorityQueue	process element in a queue
Map	HashMap, HashTable	store key-value pair

1) Array List

- ArrayList is a class in java that store element in a dynamically resizable array.
- It belong to the java.util package
- It can store duplicate element
- It size increases or decreases dynamically.
- It allows accessing elements using an index

Operation

- Add elements :- To add a single element to the arraylist, we use the add() method of the Arraylist class
- Access elements: To access an element from the arraylist, we use get() method of the Array list
- change element: To change element of the arraylist, we use set() method of the Arraylist class
- Remove element: To change an element from the arraylist, we can use the remove() method of the Array list.

Syntax:

```
Arraylist<type> arraylist = new Arraylist<>();
// create integer type Arraylist
Arraylist<Integer> ll = new Arraylist<>();
// create String type arraylist
Arraylist<String> arraylist = new Arraylist<>();
```

Program:

```
import java.util.Arraylists
class Main
{
public static void main(String[] args)
{
Arraylist<String> languages = new Arraylist<>()
languages.add("java");
language.add("python");
system.out.println("ArrayCnt:" + language));
}
}
```

Method	Descriptions
size()	Return the length of the arraylist
sort()	sort the arraylist element
clone()	creates a new arraylist with same element, size and capacity.
isEmpty()	checks if the arraylist is empty

2) Vector

- vector is a class in java that stores elements in a dynamically resizable array. It is synchronized, so its individual methods are thread safe
- It belongs to the java.util package
- It allows duplicate elements
- It maintains insertion order
- Its size increases or decreases dynamically
- It is synchronized and supports multiple threads safely when using its methods.

Operations:-

- Insertion: Add elements using add()
- Deletion: Remove elements using remove()
- Accessing: Retrieve elements using get()
- Updating: change elements using set()
- Searching: search for elements using contains()

Syntax:-

```
Vector <Type> vector = new Vector <>();
```

Method:-

- add(): Add an elements
- remove(): Remove an elements
- get() : Retrieves an element
- set() : Updates an element
- size() : return the number of elements
- firstElement(): return the first element

Program:-

```
import java.util.*;
class Main
{
public static void main(String[] args)
 {
    Vector <Integer> v = new Vector <>();
            v.add(10);
            v.add(20);
           System.out.println(v);
           System.out.println(v.get(1));
           v.remove(0);
           System.out.println(v);
    }
}
```

3) Treeset

Treeset is class in java that stores unique elements in sorted order.

- It belongs to the java.util Package
- It does not allow duplicate elements.
- It stores element in ascending order by default
- It is based on a tree structure

~~operati~~

Method:-

- add():- Add an element
- ~~put(): Add a key-value pair~~
- remove : Removes an element
- first(): returns the first element
- last(): returns the last element
- contains(): checks whether an element exists.
- size() : returns the number of elements
- clear(): removes all elements

Syntax:-

```
Treeset <Integer> number = new Treeset<>();
```

Program

```
import java.util.*;
Public Static void main(String[] args)
{
Treeset < Integer > t = new Treeset <>();
        t.add(30);
        t.add(10);
      System.out.println(t);
}
```

4) Hash map

Hashmap is class in java that stores data in key-value pair

- It belongs the java.util package.
- It stores data as key-value pairs
- It does not allow duplicate keys
- It allows duplicate values.
- It does not guarantee the order of elements

Method:-

- put() :- Add a key-value pair
- get() :- Retrieves a value using a key
- remove() :- Removes a key-value pair
- containsKey() :- checks whether a key exists
- containsValue() :- Check whether a value exits

Syntax:

```
HashMap<Integer, String> h = new HashMap<>();
```

Program

```
import java.util.*;

class Main
{
public static void main(String[] args)
{
HashMap<Integer, String> h = new HashMap<>();
        h.put(1, "Apple")
        h.put(2, "Mango")
    System.out.println(h)
    System.out.println(h.get(2));
}
}
```

Hashtable

- java Hashtable class is synchronized
- java Hashtable class doesn't allow null key or value
- A Hashtable contains values based on the key - Java Hashtable class contains unique elements
- Java Hashtable class implements a ~~hash~~ hashtable, which maps keys to values.
- It inherits Dictionary class and implements the Map interface.

Constructor	Description
Hashtable()	It creates an empty hashtable having the initial default capacity and load factor
Hashtable(int capacity)	It accepts an integer parameter and creates a hashtable that contain a specified initial capacity
Hashtable(int capacity, float loadfactor)	It is used to create a hashtable having specified initial capacity and load factor
Hashtable(Map<? extends K, ? extend V> t)	It creates a new hashtable with the same mappings as the given map.

some program on Hashtable

Iterator :- Iterator is a interface in java used to access element of a collection one by one

method:-

1. hasNext() :- Checks whether the next element exists
2. next() :- Returns the next element
3. remove() :- Removes the current element

Program:-

```
import java.util.*;
class Main
{
public static void main(String[] args)
{
ArrayList<String> a = new ArrayList<>();
a.add("Apple");
a.add("Mango");
Iterator<String> i = a.iterator();
while(i.hasNext())
{
System.out.println(i.next());
}
}
}
```

Comparator

- Comparator is an interface in java used to define custom sorting order for objects.
- Comparator interface is used to order the objects of a user-defined class
- A comparator object is capable of comparing two object of the same class

Syntax:-

public int compare(Object obj1, Object ob2)

method :-

.compare() :- Compares two objects

.reversed() :- reverses the Comparator's ordering

.thenComparing() :- Adds another sorting condition

Unit-V

Applet, AWT And Swings

• Applet class

An applet is a java class used to create small programs that run inside a web browser or a applet viewer.

• Applets are now obsolete and unsupported by modern browsers

Features of Applet class

- The applet class belongs to the java.applet package
- It is extended using the "extends Applet" keyword
- It does not use the main() method
- It uses a lifecycle to control the execution of the applet
- It can display text, graphics and simple user interface.

Method :-

- init() :- Initializes the applet. it is called only once.
- start() :- Starts or resumes the applet
- paint(Graphics g) :- Display text, images and graphics on the screen
- stop() : Stops the applet when it becomes inactive
- destroy() :- Release resource when the applet is closed

Program :-

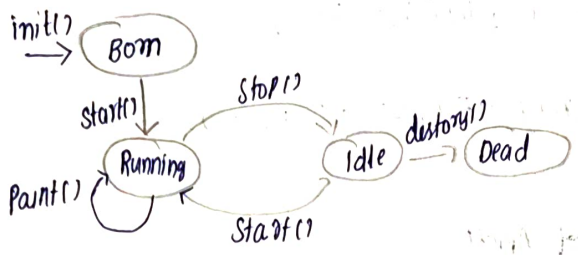
```
import java.applet.*;
import java.awt.*;
Public class MyApplet extends Applet
{
public void paint(Graphics g)
{
g.drawString("Hello World", 50, 50);
}
}
```

Structure of Applet

```
import java.applet.*;
import java.awt.*;
public class MyApplet extends Applet
{
public void init()
{
}
public void start()
{
}
public void paint(Graphics g)
{
g.drawString("Hello java", 50, 50)
}
public void stop()
{
}
public void destroy()
{
}
}
```

Applet life cycle

• The life cycle of a java applet refers to the various stages an applet goes through from its initialization to its termination.



1) Initialization:-

- Class loading: The applet's class is loaded by the Java Virtual Machine
- The init() method is invoked to initialize the applet. This method is called once in the lifecycle of an applet

Syntax:- public void init()

2) Starting:

Start Method: The start() method is called after the init() method to indicate that the applet is starting executing.

Resuming:- If an applet is stopped, it can be restarted using the start() method

Syntax:- public void start()

3) Running state

- Running state:- the applet remains in this state as long as it is visible on the web page.
- During this phase, user can interact with the applet, and the applet can process events and updates its displays as required.

4) Stopping:-

stop Method:- The stop() method is called when the applet is no longer visible on the screen or when the web page containing the applet stop.

- In this method, you can release resources, stop threads, or perform any cleanup activities necessary when applet stops

syntax:- public void stop()

5) Destroying

- the destroy() method is invoked when the applet is about to unloaded from memory

syntax: public void destroy()

Program

```
import java.applet.Applet;
import java.awt.Graphics
public class AppletLife extends Applet
{
public void init()
{
System.out.println("Initialized Applet");
}
public void start()
{
System.out.println("starting Applet");
}
public void paint(Graphic g)
{
g.drawString("Applet life", 50, 50)
}}
```

Event handling :-

Introduction

Windows programming is based on event handling mechanism. Event handling is a mechanism where program execute based on event performed. for example, let us consider operations.

1. When mouse click is performed on the mycomputer object of desktop, the object is immediately gets selected.
2. When mouse righ-click is performed on mycomputer object, a short cut menu is displayed
3. When mouse double-click is performed on mycomputer object, open the mycomputer window.

* In above three operation, mycomputer is the object of source click, right-click and double-click are "event". Select of object, displaying short cut menu and opening of mycomputer window are the "programs"

Event Source :-

An event source is an object that generates event. A single source can generate more than one event. In the above example, mycomputer is an event source. Other event source are : mouse, keyboard, button, checkbox etc

Event :

A event is an object that describes a state change in a source. In the above example mouse click, mouse right-click, are event. Other event are : keydown, keyup, actionevent etc.

Event Listeners

- A listener is an object that is notified when an event occurs.
- Listeners are the interface that receives and handles the event.
- Events are received by the listeners and it is the one which call the handlers to handle the event

Ex :- MouseListener, KeyListener etc.

Handlers

- Handlers are the methods that are executed from the event.
- These handlers are the actual ones that perform the operations for the event.

Ex :- mouseClicked(), keyDown() etc.

Event Object

- The superclass of all event classes is java.until.EventObject.
- This class contains two methods getSource() and toString().
- The getSource() method return object on which the event initially occured

Type of Events

Event Type	Description
1. ActionEvent	Occurs when a button is clicked or a menu item is selected
2. MouseEvent	Occur when the mouse is clicked, pressed or released
3. KeyEvent	Occur when a keyboard key is pressed, released, or typed
4. WindowEvent	Occur when a window is opened, closed, activated or deactivated.
MouseMotionEvent	Occur when the mouse is moved or dragged

Type of Event Listeners

Listener	Purpose
ActionListener	Handles action events
MouseListener	Handles mouse click and button event
mouseMotionListener	Handles move movement and dragging
KeyListener	Handles keyboard event
WindowListener	Handles window events

- Event Delegation Model
- The Event Delegation is a mechanism in java in which an event source generates an event and delegates its handling to a registered event listener.
- It is a technique of delegating the event-handling mechanism to a separate class.

Components

- Event Source:- The component that generates an event
  Ex:- Button
- Event object:- Contains information about the event
  Ex:- ActionEvent
- Event Listener:- Receives the event and responds to it.
  Ex:- ActionListener
- Event Handler:- The code that executes when the event occurs.

Explanation

- In the event-handling program, the applet user interface code and event-handling code are written in the same class
- As the program size grows, this type of programming become inconvenient.
- To overcome this problem, we separate the applet user interface code from the event-handling code
- The event-handling code is written in a separate listener class
- When an event occurs in the applet, it is received by the listener, which calls the handler present in the listener class.
- After executing the handler, control goes back to the applet
- This technique is known as the Delegation Event model

Program:-

```
import * java.applet.*;
import java.awt.*;
import java.awt.event.*;
Public class MouseDemo extends Applet
{
  public void init()
  {
    addMouseListener(new MyMouse());
  }
}
Class MyMouse extends MouseAdapter
{
  Public void mouseClicked(MouseEvent e)
  { System.out.println("Mouse Clicked");
}}
```

Advantage

- Separates user interface code from event-handling code.
- Makes programs easier to maintain
- provides a modern approach to handling event in window programming.

Interfaces

- Mouse Interfaces

- Mouse interfaces are used in java to handle mouse-related event in GUI applications. They allow user to interact with components using a mouse.

There are two main mouse interfaces:

1. MouseListener:-

mouse listener is an interface used to handle mouse actions such as clicking, pressing and releasing.

Method

1. mouseClicked():- called when the mouse is clicked
2. mousePressed():- called when the a mouse button is pressed.
3. mouseEntered():- called when the mouse enters a component
4. mouseExited():- called when the moves leaves a component

2. MouseMotionListener:

mouse motion listener is an interface used to handle mouse movement activities

1. mouseMoved():- called when the mouse pointer moves
2. mouseDragged:- Called when the mouse is moved while holding a mouse button

ActionListener

- ActionListener is an interface in java used to handle action events, such as clicking a button

method

actionPerformed(ActionEvent e)

- It is called whenever an action event occur
- It executes the code when the user click a button

Methods

- keyPressed(): called when a key is pressed
- keyReleased(): called when a key is released
- keyTyped(): called when a character is type

Java.awt.event Package

The java.awt.event package in java is used to handle events generated by user interaction such as mouse clicks, key presses and button click.

- Feature

* It is used for event handling in java
- It handles user action like mouse clicks and keyboard presses
- It supports the event Delegation model
- It contains event classes and listener interface

Event Class	Description
Event Class	
ActionEvent	occurs when a button is clicked

- MouseEvent :- Occurs when the mouse is used
- KeyEvent :- Occur when the key is pressed

Syntax:- :. Interface write like Action in exam

```
import java.awt.*;
import java.awt.event.*;
```

program from lab

Adapter classes

- Adapter classes are the predefined classes present in java.awt.event package
- Adapter classes implements from its respective listener interface and overrides all abstract methods as dummy methods.

Ex:- class MouseAdapter implements MouseListener

```
{
  Public void mouseClicked(MouseEvent m) { }
  public void mouseEntered(MouseEvent m) { }
}
```

Types of Adapter Class

Adapter Class	Purpose
MouseAdapter	Handles mouse events
KeyAdapter	Handles keyboard event
WindowAdapter	Handles window event
Focus Adapter	Handles focus events

KeyAdapter

- KeyAdapter is a adapter class in java used to handle keyboard events. It
- It allow us to override only the required keyboard method instead of implements all methods of the KeyListener interface.

method:

1. keyPressed()
2. keyReleased()
3. keyTyped

Program:-

```
import java.awt.*;
import java.awt.event.*;

class Demo extends Frame
{
Demo()
{
addKeyListener(new KeyAdapter()
{
public void keyPressed(KeyEvent e)
{
System.out.println("key pressed");
}
});
SetSize(300,200);
setVisible(true);
}
public static void main(String[] args)
{
new Demo();
}
}
```

MouseAdapter

- MouseAdapter is an adapter class in java used to handle mouse events. it allows us to override only the required mouse methods. instead of implementing all methods of the mouseListener interface

method:-

mouseClicked, mousePressed, mouseReleased, mouseEntered, mouseExited

Program:-

```
import java.awt.*;
import java.awt.event.*;
class Demo extends Frame
{
Demo()
{
addMouseListener(new MouseAdapter())
{
public void mouseClicked(MouseEvent e)
{
System.out.println("mouse Clicked");
}
});
SetSize(300,200)
SetVisible(true)
}
public static void main(String[] args)
{
new Demo()
}
}
```

- MouseEvent. MouseEvent is a class in the java.awt.event package used to handle mouse actions such as clicking, pressent, and releasing

method:- mouseClicked(), mousePressed(), mouseReleased()

- KeyEvent:-

key is a class in the java.awt.event package used to handle keyboard actions such as pressing, releasing and typing keys.

method:-

KeyPressed(), keyReleased(), keyTyped()

- ActionEvent:

ActionEvent is a class in the java.awt.event package. generated when the user performs an action, such as clicking a button.

method:- actionPerformed()

AWT [Abstract Window Toolkit]

- It is a java package used to create Graphical User Interface, such as windows, buttons, labels, textfields and menus.
- It supports event handling
- It provides layout to arrange components
- It is platform-dependent because it uses native operating system component

Syntax:-

```
import java.awt.*;
```